Architecture Big Data appliquée à l'IA

Séance 2 : architecture lakehouse

Séance 2 : architecture lakehouse

Support de cours (2 h), fil conducteur ShopFlow.
Objectif : comprendre ce que le lakehouse corrige dans le couple « data lake + data warehouse », comment un format de table open source apporte des transactions sur un stockage objet, et comment organiser les données en couches bronze, silver et gold.

1. Du data warehouse au data lake, puis au lakehouse

1.1 Les limites du data warehouse seul

- Le data warehouse de la séance 1 stocke les données dans un **format propriétaire**, lisible uniquement par son moteur SQL.
- Il gère mal les données **non structurées ou semi-structurées** (logs, JSON, images) et les usages **non SQL** (Python, machine learning).
- Le stockage y est plus cher qu'un stockage objet brut, et chaque nouvel usage impose une copie.

1.2 Le data lake : tout garder, à bas coût

- Idée apparue avec Hadoop (années 2010) : déposer **tous les fichiers bruts** (CSV, JSON, Parquet) dans un stockage distribué bon marché, aujourd'hui un stockage objet (S3, GCS, ADLS).
- **Schéma à la lecture** (*schema-on-read*) : on ne valide rien à l'écriture, on interprète à la lecture.
- Problèmes rencontrés : pas de transactions (un lecteur peut voir un fichier à moitié écrit), pas de mise à jour ni de suppression simple, pas de contrôle de schéma, performances médiocres. Faute de gouvernance, le lac devient un « marécage de données » (*data swamp*).

1.3 L'architecture à deux étages et ses coûts

- Pendant des années, les entreprises ont combiné les deux : un data lake pour tout stocker, puis un **ETL vers un warehouse** pour la BI.
- Conséquences : données copiées deux fois, **données périmées** dans le warehouse (le temps de l'ETL), incohérences entre les deux systèmes, coût double, et le ML travaille sur le lac sans les garanties du warehouse.

1.3 L'architecture à deux étages et ses coûts (suite)

Architecture à deux étages : lac + warehouse

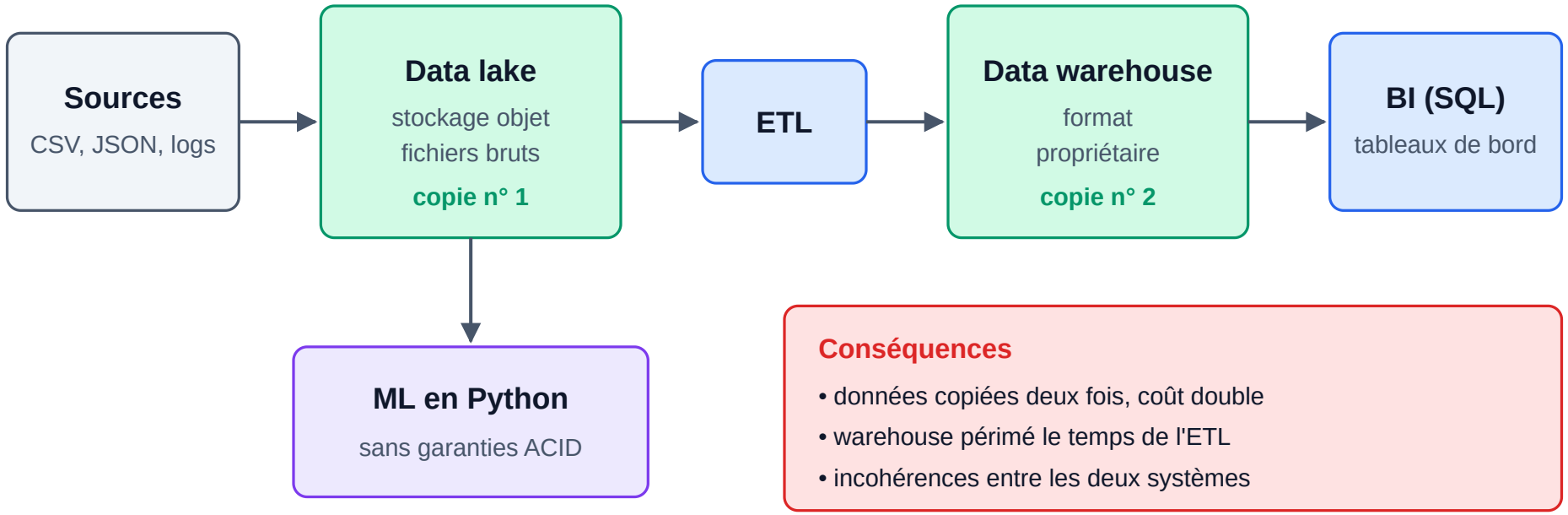


Schéma : les mêmes données sont stockées deux fois (lac puis warehouse), et le ML lit le lac sans garanties transactionnelles.

1.4 Le lakehouse

- Un système de gestion de données fondé sur un **stockage objet et en format open source**, auquel on ajoute les fonctions d'un warehouse : **transactions ACID** (définies juste après), **versions, index et cache, optimisation des requêtes**.
- Les données restent dans des fichiers au format open source (Parquet) que tout moteur peut lire ; une **couche de métadonnées transactionnelle** au-dessus décrit quels fichiers forment la version courante de chaque table.

1.4 Le lakehouse (suite)

Lakehouse : une seule copie, plusieurs usages

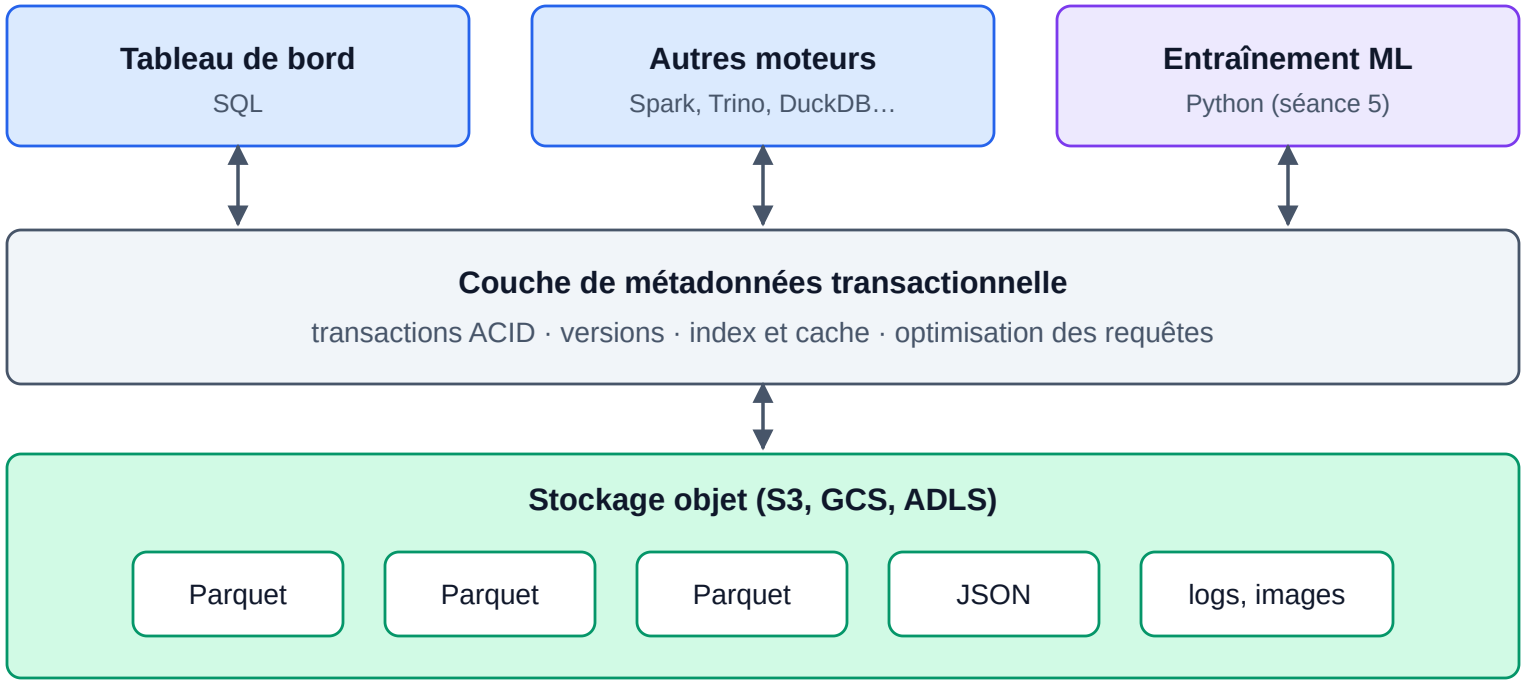


Schéma : une seule copie des données en format open source ; c'est la couche de métadonnées qui apporte les fonctions du warehouse.

Rappel : transaction et propriétés ACID

- Une **transaction** est un groupe d'opérations qu'une base de données traite comme **un seul bloc**. Exemple du TP : valider une commande, c'est (1) enregistrer la commande, (2) enregistrer le paiement, (3) retirer l'article du stock. Les trois doivent réussir ensemble, ou aucune.
- **ACID** résume les quatre garanties qu'une base offre sur chaque transaction :

Lettre	Garantie	En clair	Exemple du TP
A	**Atomicité**	Tout ou rien : si une étape échoue, la base annule celles déjà faites.	Le serveur tombe après l'étape (1) : la commande est annulée, pas de commande sans paiement.
C	**Cohérence**	La transaction fait passer la base d'un état valide à un autre état valide : les règles déclarées (contraintes) sont toujours respectées.	Le stock ne devient jamais négatif ; un paiement pointe toujours vers une commande qui existe.
I	**Isolation**	Deux transactions lancées en même temps ne se gênent pas : chacune ne voit pas le travail à moitié fait de l'autre.	Deux clients achètent le dernier article à la même seconde : un seul l'obtient.
D	**Durabilité**	Une fois la transaction confirmée (*commit*), le résultat n'est plus perdu, même en cas de panne.	Le client a reçu « commande validée » : une coupure de courant juste après ne l'efface pas.

Rappel : transaction et propriétés ACID (suite)

- **Le rôle dans une base de données** : ACID décharge le développeur de deux problèmes difficiles, les **pannes** en plein milieu d'une écriture et les **accès simultanés**. Sans ces garanties, chaque programme devrait vérifier lui-même qu'il ne laisse pas de données à moitié écrites ou écrasées par un autre.
- **Comment la base s'y prend**, en résumé : elle note chaque modification dans un **journal** sur disque avant de l'appliquer, ce qui permet d'annuler (A) ou de rejouer après une panne (D) ; elle utilise des **verrous** ou des **versions** des données pour isoler les transactions (I) ; elle vérifie les **contraintes** déclarées (C). Rappel de la séance 1 : la RAM s'efface à la coupure, c'est pourquoi la durabilité passe par le disque (ou par des copies sur plusieurs machines).

Rappel : transaction et propriétés ACID (suite)

- **Nuances utiles** : la cohérence dépend aussi de l'application (la base ne vérifie que les règles qu'on lui a déclarées) ; et beaucoup de bases offrent par défaut une isolation **moins stricte** que « comme si les transactions passaient l'une après l'autre », par exemple l'isolation par **instantané** (*snapshot*) : chaque lecture voit une photo figée de la base. C'est celle de Delta Lake (section 2.3).
- **Pourquoi c'est le cœur du sujet** : un data warehouse offre ACID ; un data lake (de simples fichiers sur un stockage objet) non. Un job Spark qui plante au milieu y laisse des fichiers à moitié écrits, et deux jobs simultanés peuvent s'écraser. Le lakehouse réintroduit ACID sur le stockage objet grâce à un **journal de transactions** (section 2).

Rappel : transaction et propriétés ACID (suite)

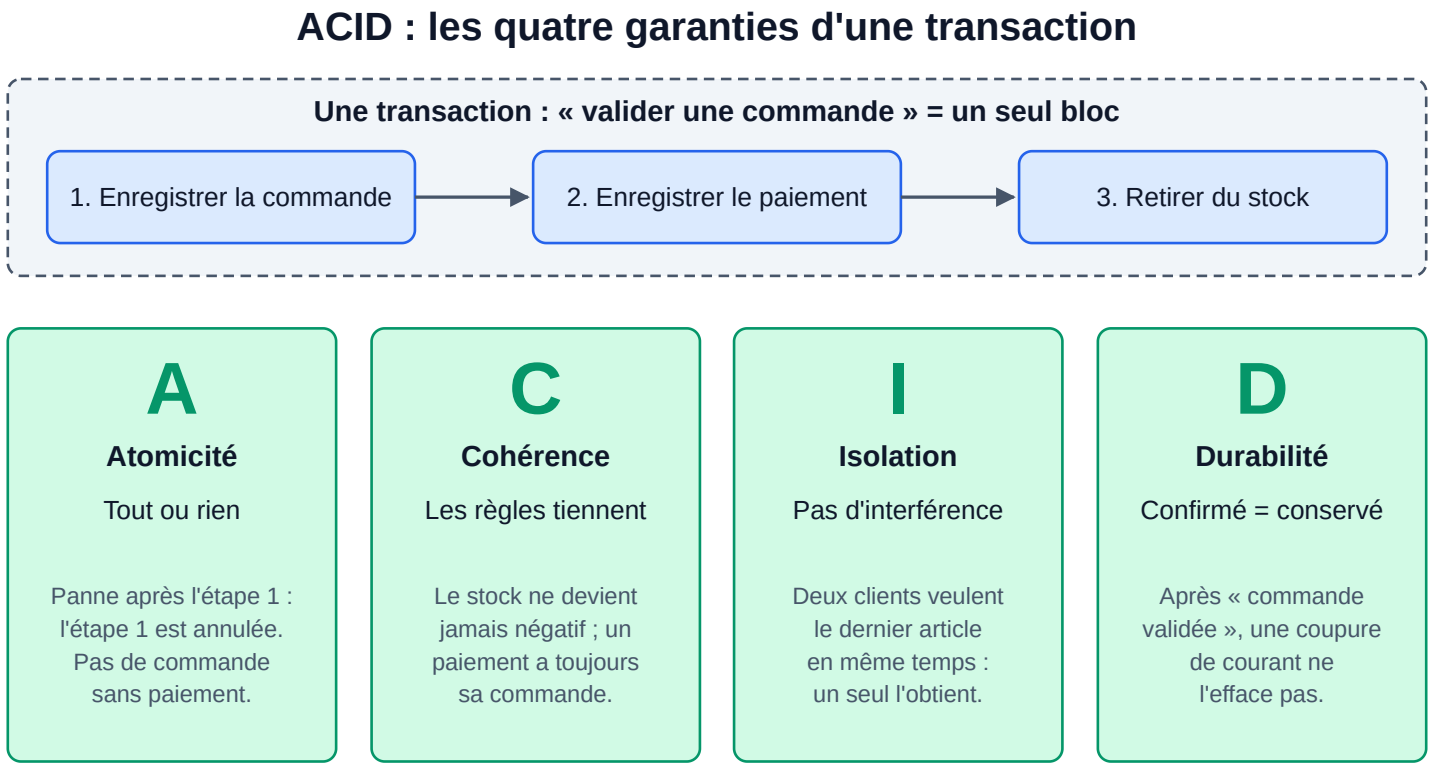


Schéma : une transaction regroupe plusieurs opérations ; ACID garantit qu'elle s'applique en entier ou pas du tout, sans règle violée, sans interférence entre clients, et sans perte après confirmation.

2. Formats de table open source

2.1 Le problème à résoudre

Un stockage objet n'offre ni renommage atomique de dossier, ni transaction sur plusieurs fichiers. Comment garantir qu'un lecteur voie **soit toute** une écriture, **soit rien** ?

2.2 La réponse : un journal de transactions

- Delta Lake ajoute à côté des fichiers Parquet un dossier `_delta_log/` contenant des fichiers JSON numérotés (`000…000.json`, `000…001.json`…). Chaque fichier est un **commit** qui liste les fichiers de données **ajoutés** et **retirés**.
- La version courante de la table = l'ensemble des fichiers ajoutés et non retirés, en rejouant le journal. Des *checkpoints* Parquet résument périodiquement le journal pour accélérer la lecture.
- Un commit réussit seulement si le fichier de journal suivant n'existe pas encore : c'est un **contrôle de concurrence optimiste**. Deux écrivains en conflit : l'un gagne, l'autre réessaie.
- Le journal stocke aussi les **statistiques par fichier** (min, max, nombre de nulls) utilisées pour sauter des fichiers (*data skipping*, vu en séance 1).

2.2 La réponse : un journal de transactions (suite)

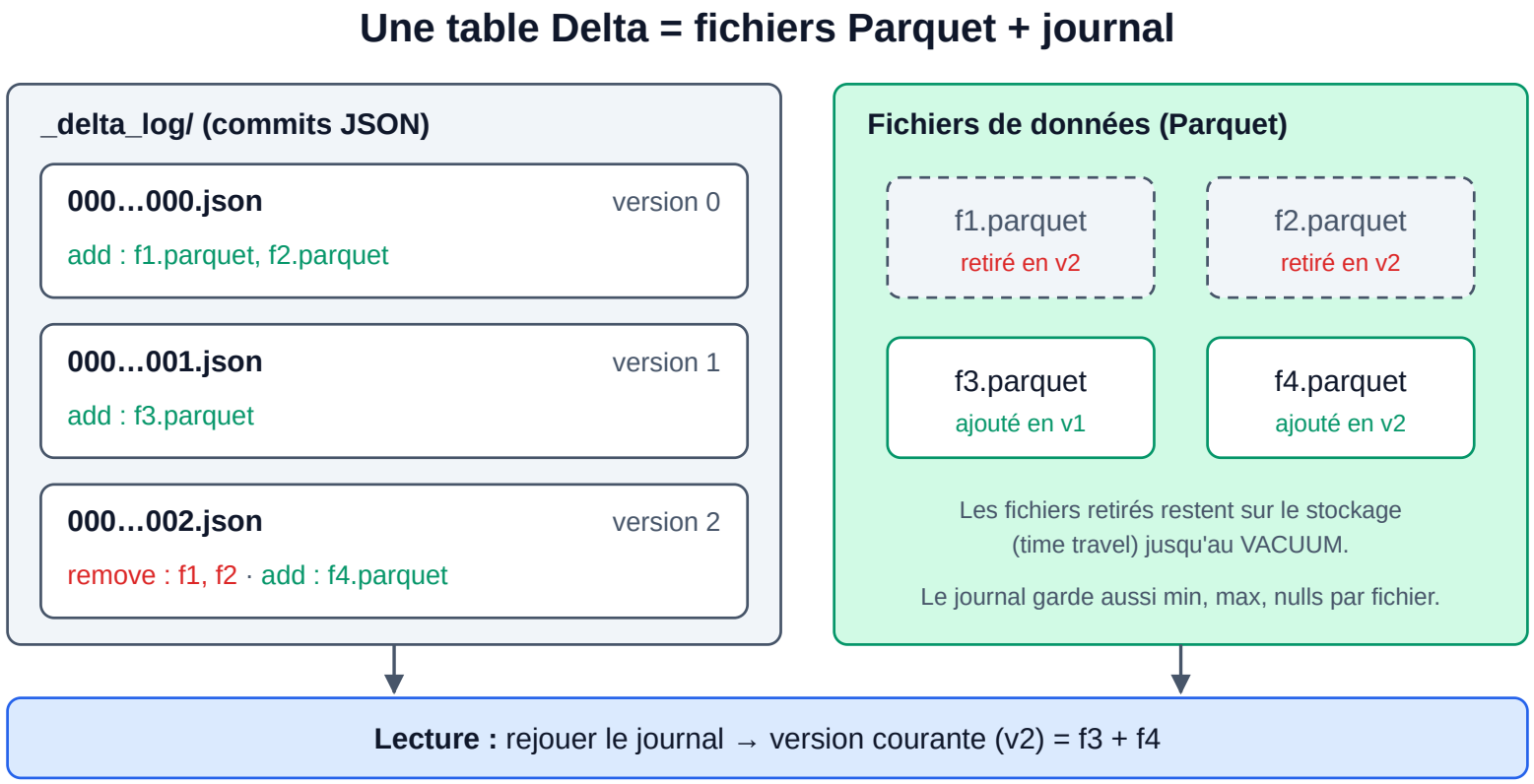


Schéma : rejouer les commits 0, 1 et 2 donne la version courante (f3 + f4) ; f1 et f2, retirés, restent sur le stockage.

2.2 La réponse : un journal de transactions (suite)

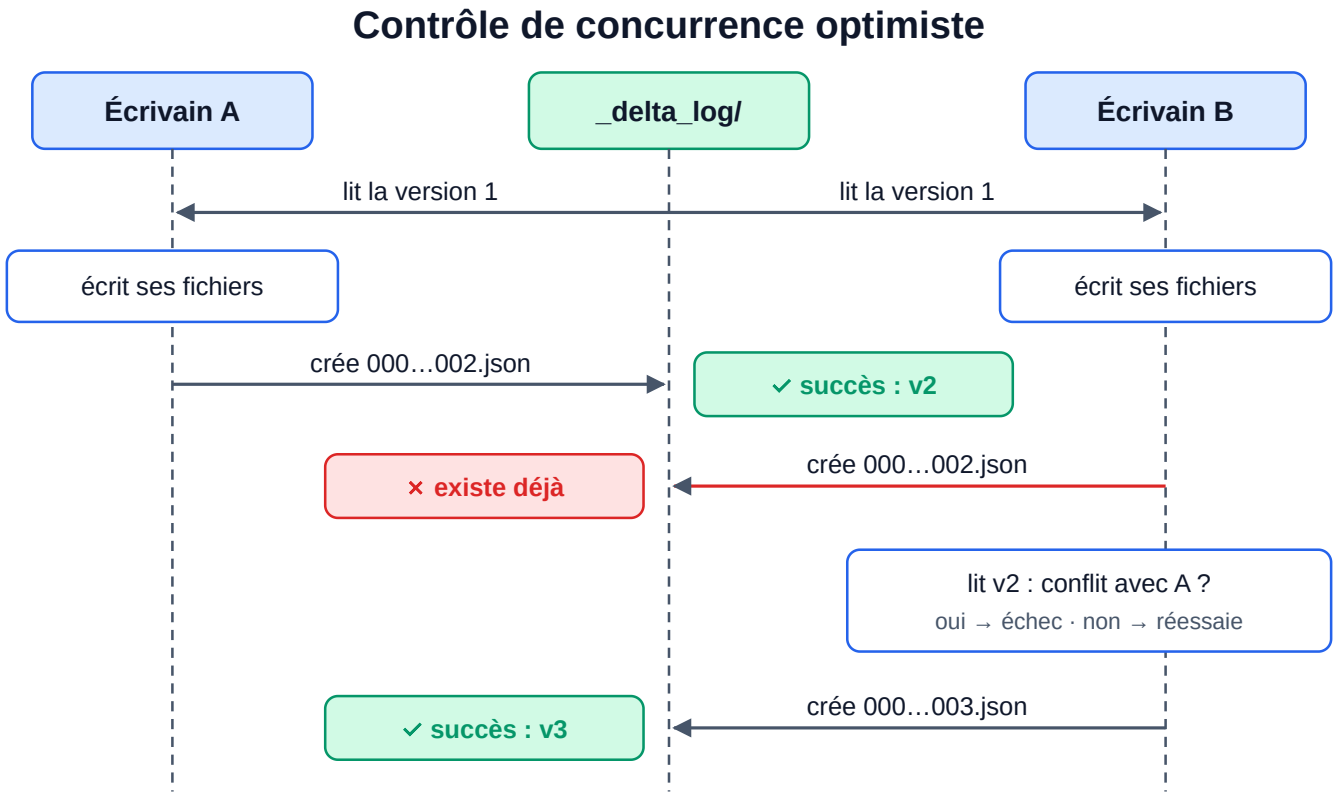


Schéma : la création du fichier de commit suivant sert de verrou ; le perdant relit la nouvelle version avant de réessayer.

2.3 Ce que cela permet

- **ACID** sur stockage objet : écritures atomiques, lectures cohérentes (isolation par instantané).
- `UPDATE`, `DELETE`, `MERGE` : on réécrit les fichiers concernés et on commit l'échange.
- **Time travel** : relire la table telle qu'elle était à une version ou une date passée (`VERSION AS OF`, `TIMESTAMP AS OF`), utile pour l'audit et la reproductibilité d'un entraînement ML.
- **Contrôle et évolution du schéma** : une écriture au mauvais schéma est rejetée, sauf évolution explicite.
- **Streaming et batch sur la même table**.

2.3 Ce que cela permet (suite)

UPDATE : réécrire des fichiers, puis commiter

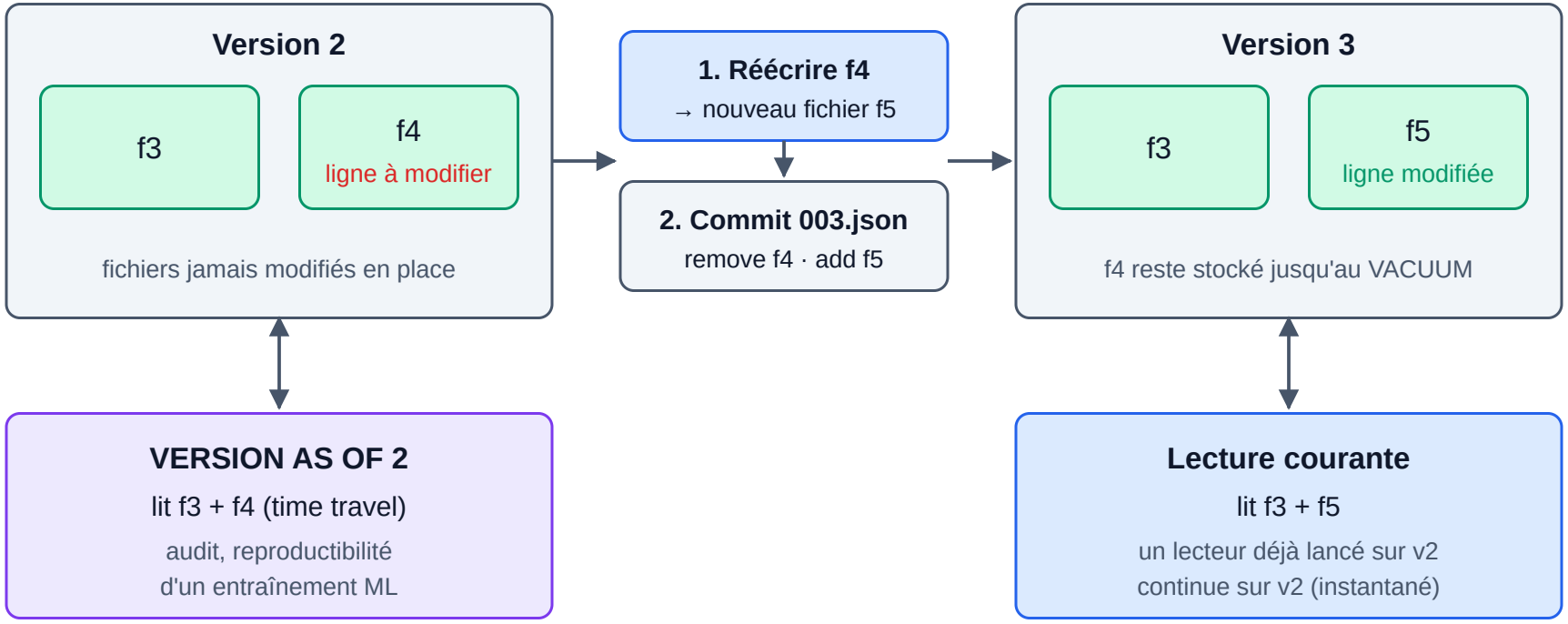


Schéma : rien n'est modifié en place ; comme l'ancien fichier reste stocké, l'ancienne version reste lisible (time travel).

2.4 Trois formats concurrents

	Delta Lake	Apache Iceberg	Apache Hudi
Origine	Databricks (2019), Linux Foundation	Netflix (2017), fondation Apache	Uber (2016), fondation Apache
Métadonnées	Journal de commits JSON + checkpoints	Arbre : fichier de métadonnées, listes de manifestes, manifestes	Timeline d'actions + index
Point fort	Intégration Spark/Databricks, simplicité	Indépendance vis-à-vis du moteur, évolution de partitionnement cachée	Mises à jour fréquentes (*upserts*), tables *merge-on-read*
Moteurs	Spark, Trino, Flink, DuckDB, Polars…	Spark, Trino, Flink, Snowflake, BigQuery…	Spark, Flink, Trino…

Tendance actuelle : **l'interopérabilité**. Delta UniForm expose une table Delta aussi en métadonnées Iceberg, et la plupart des warehouses (BigQuery, Snowflake, Redshift) savent lire Iceberg.

Sources : 2.4 Trois formats concurrents

Sources : Armbrust et al., *Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores*, VLDB 2020 ; Delta Lake transaction protocol ; Apache Iceberg table spec ; Apache Hudi overview ; Databricks, Delta UniForm.

3. Spark, le moteur du lakehouse

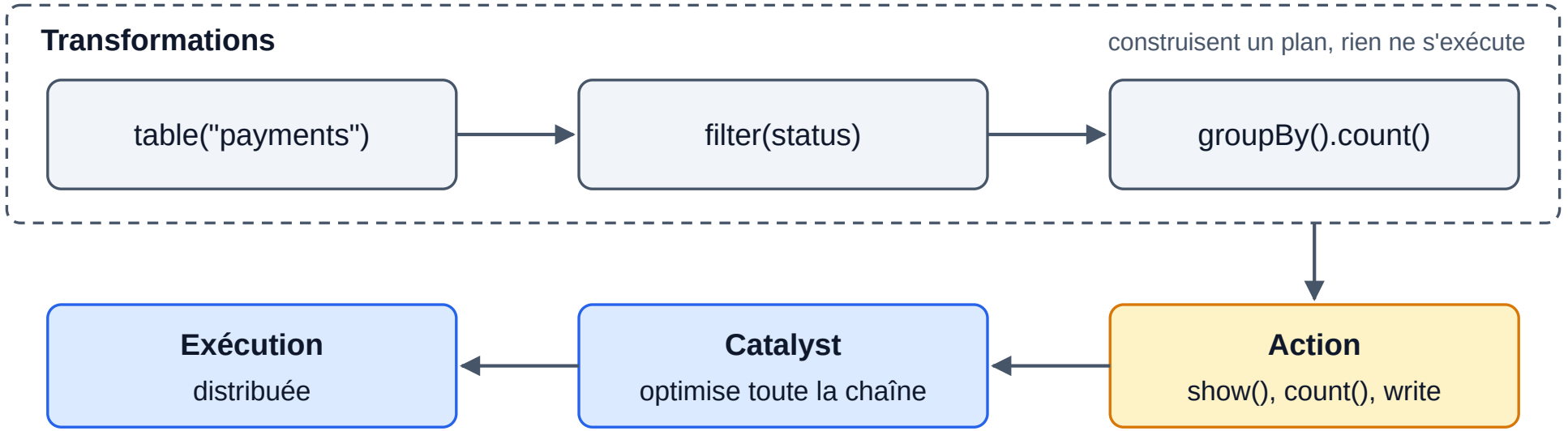
Le TP de cette séance utilise Spark ; la séance 3 en détaillera l'exécution distribuée.

3.1 Le modèle de programmation

- Une **SparkSession** est le point d'entrée. Sur Databricks, elle existe déjà sous le nom `spark`.
- Un **DataFrame** est une table distribuée, immuable, avec un schéma. On le manipule en Python (PySpark) ou en SQL : les deux produisent le même plan.
- **Transformations** (`select`, `filter`, `join`, `groupBy`) : elles décrivent un calcul sans l'exécuter.
- **Actions** (`count`, `show`, `write`, `collect`) : elles déclenchent l'exécution.
- Cette **évaluation paresseuse** permet à l'optimiseur (Catalyst) de voir toute la chaîne et de la réorganiser : pousser les filtres vers la lecture, ne lire que les colonnes utiles, choisir l'algorithme de jointure.

3.1 Le modèle de programmation (suite)

Évaluation paresseuse dans Spark



Exemples d'optimisations

filtres poussés vers la lecture · seules les colonnes utiles · choix de l'algorithme de jointure

Schéma : tant qu'aucune action n'est appelée, Spark ne fait qu'accumuler un plan que Catalyst pourra optimiser en entier.

3.2 Lire un plan

```
df = (spark.table("workspace.shopflow.payments")
        .filter("status = 'authorized'")
        .groupBy("card_country").count())
df.explain(mode="formatted")
```

On y repère : la lecture avec les filtres poussés (`PushedFilters`) et les colonnes lues, l'agrégation partielle puis l'échange (*Exchange*, le shuffle), puis l'agrégation finale.

3.2 Lire un plan (suite)

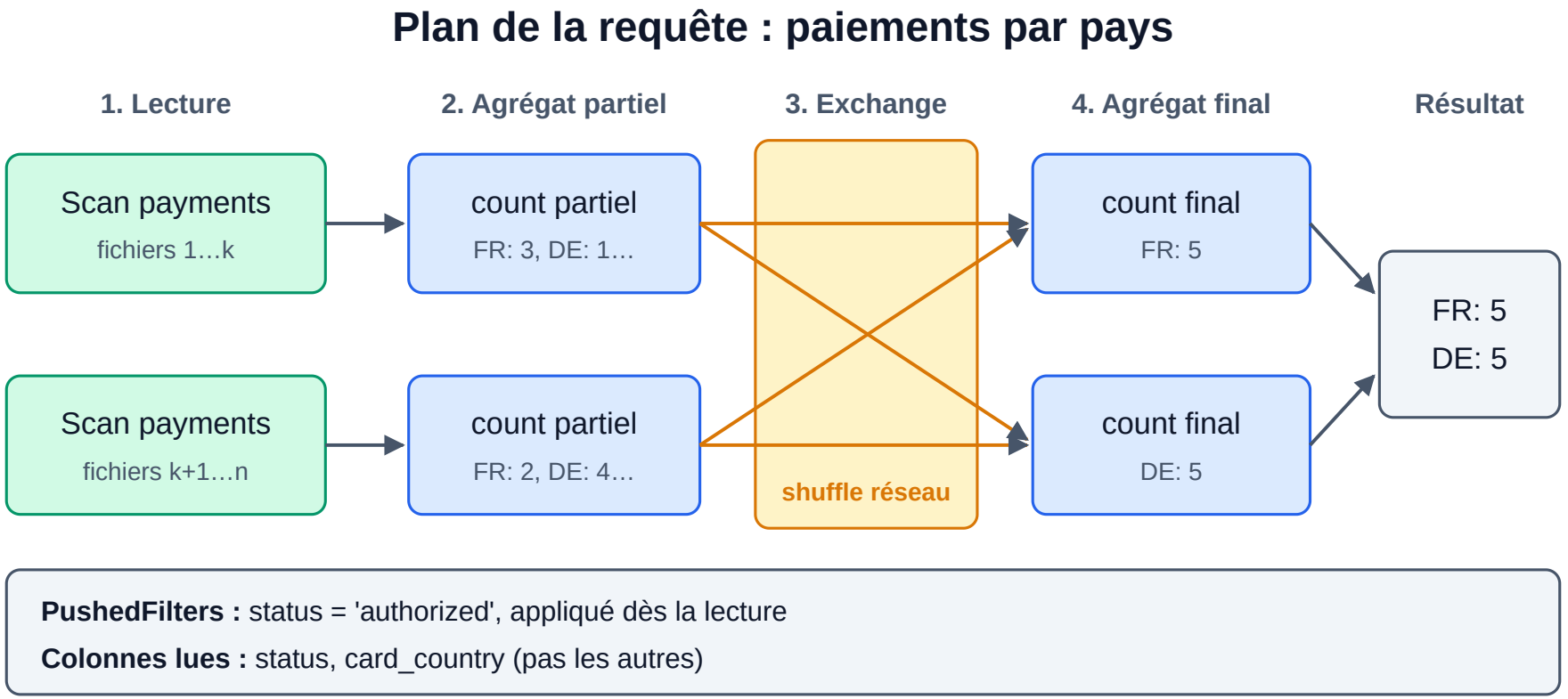


Schéma : chaque partition compte localement, le shuffle regroupe ensuite les comptes partiels d'un même pays sur une même tâche.

4. Architecture médaillon et gouvernance

4.1 Trois couches de qualité croissante

Couche	Contenu	Règle	Exemple du TP
Bronze	Données brutes, telles que reçues, avec métadonnées d'ingestion	On n'efface rien, on peut tout rejouer	Fichiers JSON du clickstream, CSV des paiements, CSV des rétrofacturations
Silver	Données nettoyées, typées, dédoublonnées, conformes	Contraintes de qualité, lignes invalides mises en quarantaine	`payments` propre, mis à jour par les rétrofacturations
Gold	Données prêtes à l'usage métier ou ML	Agrégats, features, schéma en étoile	`gold_payment_features` pour le modèle de fraude, tables du tableau de bord

4.1 Trois couches de qualité croissante (suite)

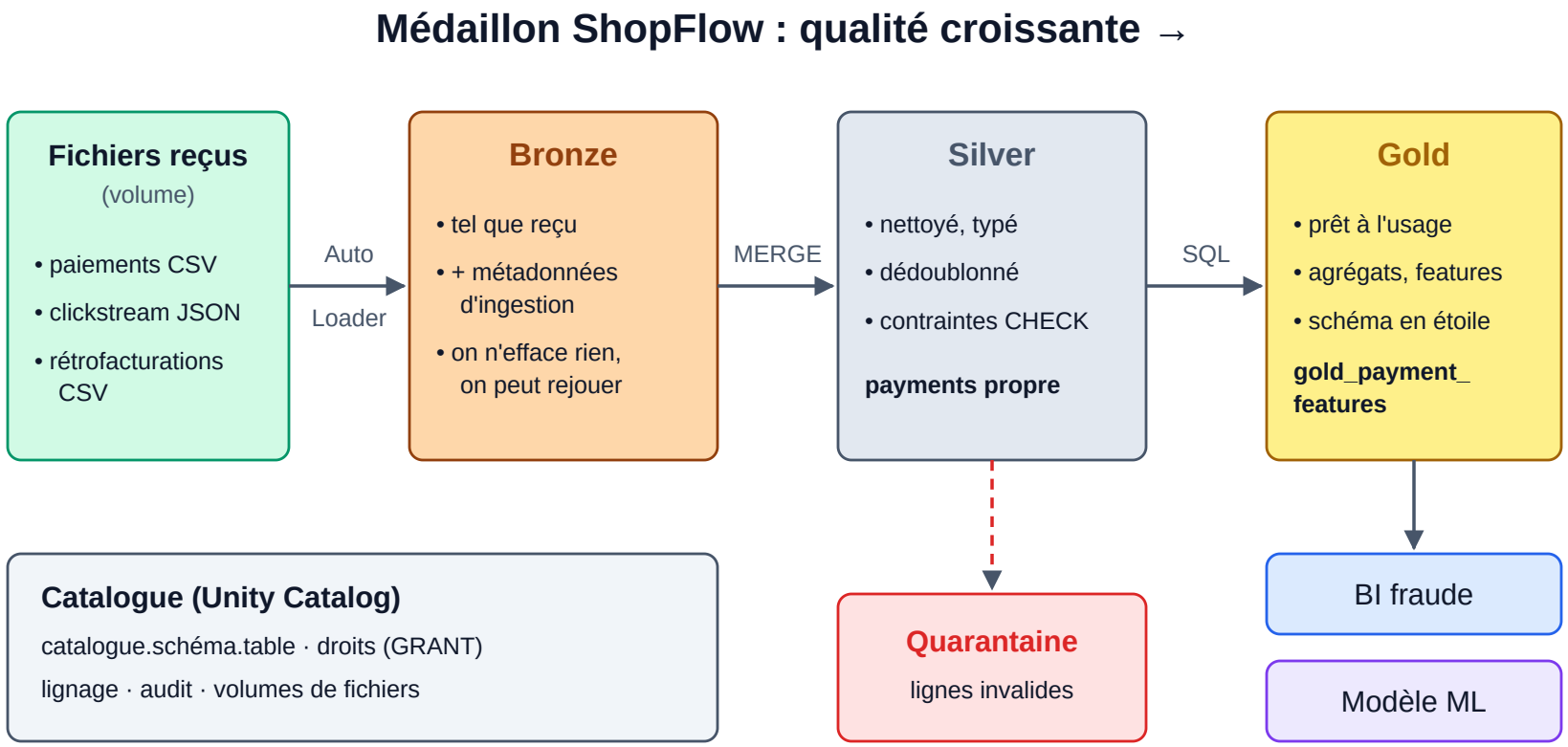


Schéma : chaque couche est une table Delta calculée à partir de la précédente ; le catalogue gouverne l'ensemble.

4.2 Principes de conception

- **Idempotence** : relancer un traitement ne doit pas dupliquer les données. On utilise `MERGE` sur une clé plutôt que `INSERT`, et des points de reprise (*checkpoints*) pour l'ingestion incrémentale.
- **Ingestion incrémentale** : ne traiter que les nouveaux fichiers (Auto Loader garde la liste des fichiers déjà vus).
- **Qualité explicite** : contraintes `CHECK` et `NOT NULL` qui font échouer une écriture invalide, ou quarantaine des lignes rejetées pour analyse.

4.3 Gouvernance avec un catalogue

- Un **catalogue** (Unity Catalog chez Databricks, open source depuis 2024) organise les objets en `catalogue.schéma.table`, gère les **droits** (`GRANT SELECT ON TABLE ... TO ...`), trace le **lignage** (quelle table est calculée à partir de quelle autre) et l'**audit**.
- Les **volumes** sont des espaces de fichiers gouvernés par le même catalogue : c'est là qu'arrivent les fichiers bronze.

Sources : 4.3 Gouvernance avec un catalogue

Sources : Databricks, What is the medallion lakehouse architecture? ; Databricks, Unity Catalog ; Unity Catalog OSS ; Databricks, Constraints on Databricks ; Databricks, Auto Loader.

5. Coût et bonnes pratiques

- **Le stockage est bon marché, le calcul est cher** : un To en stockage objet coûte de l'ordre de 20 $ par mois (tarifs Cloud Storage) ; le coût d'un lakehouse se joue surtout sur le calcul.
- **Le problème des petits fichiers** : une ingestion fréquente crée des milliers de petits fichiers. Chaque fichier coûte une requête au stockage objet et une entrée de métadonnées : les lectures ralentissent. Remède : `OPTIMIZE`, qui compacte les petits fichiers en fichiers plus gros et de taille homogène.
- **Organisation physique** : Databricks recommande le **liquid clustering** pour toutes les nouvelles tables, et **de ne pas partitionner une table de moins de 1 To** (chaque partition devrait contenir au moins 1 Go).
- **Rétention** : les anciennes versions restent sur le stockage pour le time travel. `VACUUM` supprime les fichiers qui ne sont plus référencés depuis la durée de rétention (7 jours par défaut) ; après, le time travel vers ces versions n'est plus possible.

5. Coût et bonnes pratiques (suite)

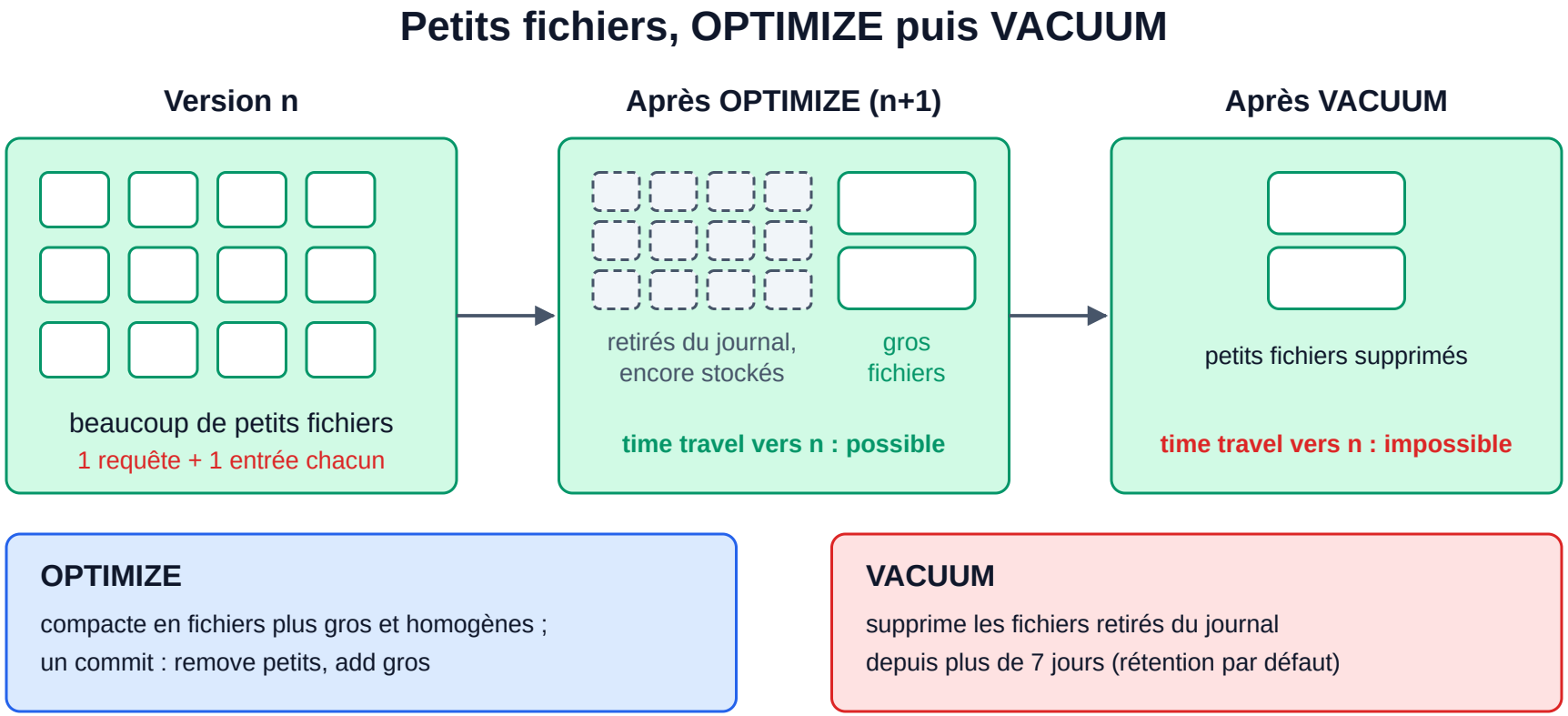


Schéma : OPTIMIZE ne supprime rien (il commite un échange de fichiers) ; c'est VACUUM qui libère le stockage, au prix du time travel.

Sources : 5. Coût et bonnes pratiques

Sources : Databricks, Delta Lake best practices ; OPTIMIZE ; VACUUM ; liquid clustering ; When to partition tables.

6. Panorama des plateformes

Plateforme	Format natif	Moteur	Particularité
Databricks	Delta (et Iceberg)	Spark + Photon	Origine du concept, Unity Catalog
Snowflake	Propriétaire + tables Iceberg	Snowflake	Tables Iceberg gérées ou externes
BigQuery	Propriétaire + tables Iceberg (BigLake)	Dremel	Le warehouse de la séance 1 lit aussi un lakehouse
Microsoft Fabric	Delta (OneLake)	Spark + SQL	Un seul lac pour toute l'organisation
Pile open source	Iceberg	Trino, Spark, Flink	Aucun éditeur, plus d'exploitation

Sources : 6. Panorama des plateformes

Sources : BigQuery tables for Apache Iceberg ; Snowflake Iceberg tables ; Microsoft Fabric OneLake ; Trino Iceberg connector.